Stretch GazeBot

Control a Hello Robot Stretch 3 with your head, using Xreal One Pro AR glasses.

Project folder	~/lab/life_home/xreal_head_control
Robot	Hello Robot Stretch 3
AR glasses	Xreal One Pro (USB-C, ethernet over USB)
PC OS	Linux (tested on Ubuntu 22.04)
Robot OS	Ubuntu 22.04 + ROS 2 Humble
Author / Owner	het@operationautopilot.com
Document version	Apr 2026

1. What this project does

Stretch GazeBot is a teleoperation bridge. You wear Xreal One Pro AR glasses on your PC. The glasses stream their IMU (gyroscope + accelerometer) data over a USB-C ethernet link to your PC. A Python program on the PC reads that stream, computes head pitch / yaw / roll, and forwards pan / tilt commands to a relay program running on the Stretch 3 robot. The relay translates those into ROS 2 joint commands for the robot's head. The result: turn your head, and the robot's head turns the same way (near real-time, < 10 ms IMU latency).

There is also a DRIVE mode, where holding SPACE makes the robot's mobile base move based on head tilt (pitch → forward / reverse, yaw → turn).

System architecture

```
[Xreal One Pro glasses]
| USB-C ethernet (TCP 169.254.2.1:52998)
v
[Your PC] stretch_head_control.py
| TCP JSON commands (port 9090)
v
[Stretch 3] stretch_relay.py --> ROS 2 follow_joint_trajectory action
--> joint_head_pan / joint_head_tilt
```

Repository layout

File	Where it runs	Purpose
stretch_head_control.py	Your PC	Main bridge. Reads glasses IMU, computes head pose, sends TCP commands.
stretch_relay.py	Stretch 3	ROS 2 node. Receives TCP commands and moves head pan / tilt.
imu_server.py / imu_viz.html	Your PC	Optional: 3D visualizer in browser to verify glasses work.
src/imu_reader_fixed.py	Your PC	Reads raw IMU stream from glasses over TCP.
src/head_tracker.py	Your PC	Sensor fusion: raw IMU → calibrated pitch / yaw / roll.
src/imu_data.py	Your PC	Dataclass holding a single IMU sample.
start.sh	Your PC	Convenience launcher — pings glasses, then runs the bridge.

requirements.txt	Your PC	Python deps: numpy, pygame, PyOpenGL (visualizer only).

2. Hardware checklist

- **Xreal One Pro** AR glasses (the regular One has not been tested and may not work).
- **USB-C cable** (data + power capable). The glasses expose a USB ethernet adapter.
- **Hello Robot Stretch 3** on the same Wi-Fi or LAN as your PC.
- **Linux PC** with Python 3.7+ (Ubuntu 22.04 recommended).
- **Network reachability**: your PC must be able to reach the robot at `172.22.243.8`. If your robot has a different IP, write it down — you will need to update `STRETCH_IP` in `stretch_head_control.py` (line 35).

3. Initial PC setup (one-time)

3.1 Clone the repo

```
mkdir -p ~/lab/life_home && cd ~/lab/life_home
git clone <your-fork-url> xreal_head_control
cd xreal_head_control
```

If you received this code as a tarball/zip instead, extract it to `~/lab/life_home/xreal_head_control` so the relative paths in the helper scripts continue to work.

3.2 Install Python dependencies

```
# (Optional but recommended) virtualenv
python3 -m venv .venv && source .venv/bin/activate

pip install -r requirements.txt
```

Note: `numpy` is the only hard requirement for the head-control bridge. `pygame` and `PyOpenGL` are only needed for the optional 3D visualizer.

3.3 Plug in the glasses and verify the link

- Connect the Xreal One Pro to your PC with USB-C.
- On the glasses, open *Settings* → *Developer Menu* → *Enable Ethernet* (only needed once).
- On your PC, confirm the link:

```
ping 169.254.2.1
```

You should see replies. If not, try a different USB-C port, re-toggle ethernet in the developer menu, or check `ip addr` for an interface with a 169.254.x.x address.

3.4 (Optional) Test the glasses on their own

```
cd ~/lab/life_home/xreal_head_control
python3 imu_server.py # in one terminal
xdg-open imu_viz.html # opens a 3D cube in your browser
```

Move the glasses — the cube should follow. Press Ctrl+C in the terminal to stop.

4. Robot setup (Stretch 3)

4.1 SSH into the robot

```
ssh hello-robot@172.22.243.8
# password: <ask lab admin>
```

Substitute your robot's IP if different. Get the robot's password from your lab admin and `change it for production use.`

4.2 Copy the relay script onto the robot

From your PC, copy `stretch_relay.py` into the robot's ROS 2 workspace:

```
scp ~/lab/life_home/xreal_head_control/stretch_relay.py \
hello-robot@172.22.243.8:~/het_ros2_ws/stretch_relay.py
```

(Create the directory on the robot first if needed: `mkdir -p ~/het_ros2_ws`.)

4.3 Start the Stretch driver (terminal A on the robot)

```
ssh hello-robot@172.22.243.8
conda deactivate # IMPORTANT — rclpy lives in the system Python
ros2 launch stretch_core stretch_driver.launch.py
```

Wait for the driver to finish initializing (you should see joint state messages).

4.4 Start the relay (terminal B on the robot)

```
ssh hello-robot@172.22.243.8
conda deactivate

# Confirm the action server name the driver advertises:
ros2 action list
# Look for: /stretch_controller/follow_joint_trajectory

# If your action server name differs, edit ~/het_ros2_ws/stretch_relay.py
# (see the import of hello_helpers.hello_misc — move_to_pose uses the standard
# Stretch action server, so usually no edit is needed).

python3 ~/het_ros2_ws/stretch_relay.py
```

When you see `Waiting for connection on port 9090...`, the robot side is ready.

5. Running the bridge on your PC

```
cd ~/lab/life_home/xreal_head_control
python3 stretch_head_control.py
# (or)
./start.sh # also pings the glasses first
```

First-run procedure:

- Place the glasses on a flat surface and keep them **completely still** for ~0.5 s while gyro calibration runs (you'll see a percentage).
- When you see *Calibration complete*, put the glasses on.
- Press **T** to set your current head pose as 'center'.
- Move your head left / right → robot pans. Tilt up / down → robot tilts.

5.1 Keyboard controls

Key	Mode	Action
M	Both	Toggle LOOK ↔ DRIVE mode
Space	DRIVE	Hold to enable base motion (head steers)
T	Both	Zero view — set current head pose as neutral
R	Both	Recalibrate gyroscope (keep glasses still)
L	Both	Lock / unlock — pause sending commands
Q	Both	Quit cleanly

6. Tuning sensitivity and direction

All tunables live near the top of `stretch_head_control.py`:

```
STRETCH_IP = "172.22.243.8" # robot IP
RELAY_PORT = 9090

HEAD_PAN_MIN = -3.9 # robot joint limits (rad)
HEAD_PAN_MAX = 1.5
HEAD_TILT_MIN = -1.53
HEAD_TILT_MAX = 0.79

HEAD_PAN_SCALE = 0.025 # bigger = more sensitive left/right
HEAD_TILT_SCALE = 0.015 # bigger = more sensitive up/down

COMMAND_INTERVAL = 0.05 # seconds between commands (20 Hz)

BASE_LINEAR_MAX = 0.3 # m/s — max forward/reverse in DRIVE
BASE_ANGULAR_MAX = 0.5 # rad/s — max rotation in DRIVE
BASE_DEADZONE = 5.0 # deg — no motion within this range
BASE_MAX_ANGLE = 30.0 # deg — angle that maps to full speed
```

If pan or tilt moves the wrong way, flip the sign in the assignment near `stretch_head_control.py` lines 163–164:

```
pan_rad = -yaw * HEAD_PAN_SCALE # flip sign of yaw
tilt_rad = -pitch * HEAD_TILT_SCALE # flip sign of pitch
```

7. Troubleshooting

Symptom	Most likely cause / fix
Cannot ping 169.254.2.1	Ethernet not enabled in glasses developer menu, or USB-C cable is power-only. Try a differe
“Connection refused” when starting the bridge	stretch_relay.py is not running on the robot. Start it first.
“Trajectory server not found” on the robot	stretch_driver.launch.py is not running, or its action server name doesn't match. Run `ros2 a
“No module named rclpy” on the robot	Conda is active and shadowing the system Python. Run `conda deactivate` first.
Robot head moves the wrong direction	Flip the sign on stretch_head_control.py line 163 or 164.
Tracking drifts after a minute	Press R to recalibrate (keep glasses still). Drift is inherent to gyro-only tracking — short sess
Too sensitive / not sensitive enough	Adjust HEAD_PAN_SCALE and HEAD_TILT_SCALE.
“T / R / Q does nothing”	Make sure the bridge terminal is focused. Some terminals require a single keypress (cbreak

8. Optional extras (used during development)

8.1 Mount the robot's ROS workspace over SSHFS

Useful for editing files on the robot from your PC:

```
mkdir -p ~/lab/life_home/shared_folder_btw_stretch3_mypc/het_ros2_ws
sshfs hello-robot@172.22.243.8:/home/hello-robot/het_ros2_ws \
~/lab/life_home/shared_folder_btw_stretch3_mypc/het_ros2_ws
```

8.2 Robot reset / homing

```
# On the robot, before starting the driver:
stretch_free_robot_process.py # release any stuck driver locks
stretch_robot_home.py # home all joints
```

8.3 Camera streams the robot exposes

```
# (each in its own SSH terminal, after `conda deactivate`)
ros2 launch stretch_core d435i_high_resolution.launch.py # head RGB-D
ros2 launch stretch_core d405_basic.launch.py # gripper cam
ros2 run v4l2_camera v4l2_camera_node \
--ros-args -p video_device:="/dev/video6" \
-r image_raw:=/arducam/image_raw # arducam
ros2 run web_video_server web_video_server # browser viewer
ros2 launch ~/het_ros2_ws/stretch_dual_camera.launch.py # combined dual cam
```

Open the dual-camera HTML viewer on your PC:

```
google-chrome --kiosk \
file:///home/het/lab/life_home/shared_folder_btw_stretch3_mypc/het_ros2_ws/dual_camera_
view.html
```

9. How the code works (for the curious)

- **IMU stream:** `src/imu_reader_fixed.py` opens a TCP connection to `169.254.2.1:52998` and decodes binary IMU frames (~1000 Hz).
- **Sensor fusion:** `src/head_tracker.py` calibrates the gyro bias from the first ~500 samples while you hold still, then integrates gyro and fuses with accelerometer using a complementary filter (96% gyro / 4% accel).
- **Mapping:** Yaw → robot pan, pitch → robot tilt, both scaled and clamped to the Stretch joint limits.
- **Transport:** Newline-delimited JSON over a single TCP socket (port 9090).
- **Robot side:** `stretch_relay.py` is a `hello_helpers.HelloNode` ROS 2 node. It calls `move_to_pose({'joint_head_pan': pan, 'joint_head_tilt': tilt}, blocking=False)` at 20 Hz.
- **DRIVE mode:** When SPACE is held, head pitch / yaw map to base `linear` / `angular` velocities with a deadzone, sent as the same JSON shape (`{linear, angular}`).

10. Safety notes

- Always keep one hand near the Stretch e-stop while teleoperating.
- Press **L** to lock tracking before adjusting the glasses on your face.
- In DRIVE mode, releasing SPACE for ~0.2 s sends a stop command automatically.

• If the network drops, the relay simply stops getting commands; the robot will hold its last commanded pose. Press the e-stop if you need it to halt immediately.

11. Quick reference (cheat sheet)

```
# 1. Robot, terminal A:
ssh hello-robot@172.22.243.8 'conda deactivate; \
ros2 launch stretch_core stretch_driver.launch.py'

# 2. Robot, terminal B:
ssh hello-robot@172.22.243.8 'conda deactivate; \
python3 ~/het_ros2_ws/stretch_relay.py'

# 3. PC:
cd ~/lab/life_home/xreal_head_control && python3 stretch_head_control.py
```

Built from `HOW_TO_USE.txt` and the project source. Credits: One Pro IMU Retriever Demo by Daniel Sami Mitwalli; Stretch integration by het@operationautopilot.com.